UNIVERSIDADE VEIGA DE ALMEIDA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GERAÇÃO DE TERRENOS 3D EM PYTHON COM URSINA

MATHEUS SOARES MARTINS
THIAGO CRIVARO NUNES

RIO DE JANEIRO
2025

UNIVERSIDADE VEIGA DE ALMEIDA

MATHEUS SOARES MARTINS
THIAGO CRIVARO NUNES

Trabalho de Conclusão de Curso apresentado no curso de Ciência da Computação da Universidade Veiga de Almeida como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Denis Gonçalves Cople

GERAÇÃO DE TERRENOS 3D EM PYTHON COM URSINA

RIO DE JANEIRO
2025

UNIVERSIDADE VEIGA DE ALMEIDA
Rio de Janeiro – RJ
Tel.: (21) 2574-8888

M344g Martins, Matheus Soares . Nunes, Thiago Crivaro .
Geração de terrenos 3D em python com ursina / por Matheus Soares Martins. Thiago Crivaro Nunes – 2025.
49 f. ; 30 cm.

Orientador: Denis Gonçalves Cople.
Trabalho de Conclusão de Curso (TCC) – Universidade Veiga de Almeida, Graduação em Ciência da Computação, Rio de Janeiro, RJ, 2025.

1. Geração procedural. 2. Ruído. 3. Python. 4. Ursina. I. Cople, Denis Gonçalves (orientador). II. Universidade Veiga de Almeida. Graduação em Ciência da Computação. III. Título.

FCBU1654/2025

Ficha Catalográfica elaborada pelo Sistema de Bibliotecas da UVA
Com dados fornecidos pelo(a) autor(a).

UNIVERSIDADE VEIGA DE ALMEIDA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

MATHEUS SOARES MARTINS
THIAGO CRIVARO NUNES

GERAÇÃO DE MUNDOS 3D EM PYTHON COM URSINA

Monografia apresentada como requisito parcial à conclusão do curso em Bacharel em Ciência da Computação.

APROVADA EM:

CONCEITO: _____

BANCA EXAMINADORA:

_____

PROF. DSc ou MSc DENIS GOLÇALVES COPLE
ORIENTADOR

_____

PROFA. DSc ou MSc CAMILLA LOBO PAULINO

_____

PROF. DSc ou MSc THIAGO GABRIEL

Coordenação de Ciência da Computação

Rio de Janeiro

"Se me oferececem a sabedoria com a condição de guardar só para mim, sem comunicar a alguém, não a quereria."

(Sêneca)

AGRADECIMENTOS

Gostaríamos de agradecer a Deus e a nossa família por sempre nos apoiar durante toda a nossa jornada, nos bons e maus momentos.

RESUMO

O presente trabalho investiga como a geração procedural de mundos virtuais funciona, focando em jogos como Infiminer e Minecraft, que possuem mundos 3D feitos inteiramente de blocos. A pesquisa analisa os algoritmos e técnicas utilizadas por esses jogos, como formas de geração de ruído, que permitem a criação de terrenos orgânicos a partir de funções matemáticas. Com base nesse estudo, então, foi desenvolvido um simulador próprio em python com a engine ursina em que é possível gerar terrenos 3D feitos com blocos, interagir com eles e criar novos com sementes de geração diferentes. O simulador utiliza o ruído gerado pelo OpenSimplex para definir a altura de cada coluna do terreno, enquanto largura e comprimento do mundo são totalmente editáveis, permitindo que terrenos de diversos tamanhos diferentes sejam gerados. Foram implementadas funções que, de acordo com a altura em que o bloco está, mudam sua textura e cor, criando um terreno com blocos de terra, grama e água, resultando em um aspecto mais natural. Também foram implementadas funções de controle em primeira pessoa que possibilitam interagir com o terreno, andando ou voando sobre ele e até mesmo quebrando blocos. O estudo permite uma compreensão melhor de como mundos complexos foram e são criados a partir de operações relativamente simples.

Palavras-Chave: Geração procedural, ruído, python, ursina, blocos

ABSTRACT

This work investigates how procedural generation of virtual worlds works, focusing on games such as Infiminer and Minecraft, which feature fully block-based 3D worlds. The research analyzes the algorithms and techniques used by these games, such as noise-generation methods that enable the creation of organic terrains from mathematical functions. Based on this study, a custom simulator was developed in python using the ursina engine, allowing the generation of 3D block terrains, interaction with them and the creation of new worlds from different seeds. The simulator uses noise generated by OpenSimplex to determine the height of each terrain column, while the width and length of the world are fully editable, making it possible to generate terrains of various sizes. Were implemented functions to change each block's texture and color according to its height, creating terrains composed of dirt, grass, and water blocks and resulting in a more natural appearance. First-person control functions were also implemented, allowing interaction with the terrain by walking or flying and even breaking blocks. This study provides a better understanding of how complex worlds have been and continue to be created from relatively simple operations.

Key-Words: Procedural generation, noise, python, ursina, blocks

LISTA DE FIGURAS

Figura 1 - Screenshot do Infiminer 17
Figura 2 - Screenshot mostrando o terreno do Infiminer 20
Figura 3 - Screenshot do Minecraft 31
Figura 4 - Ruído branco gerado por uma função aleatória 32
Figura 5 - Ruído gerado por Perlin com múltiplas oitavas combinadas 32
Figura 6 - Exemplo do mapa de água/terra gerado 33
Figura 7 - Exemplo do mapa ampliado 33
Figura 8 – Exemplo do mapa ampliado finalizado 34
Figura 9 - Gráfico com os diferentes biomas do jogo..... 34
Figura 10 - Exemplo da divisão dos biomas 35
Figura 11 - Representação de um chunk do minecraft 37
Figura 12 – Terreno 10x10x10 43
Figura 13 – Terreno 10x10x10 refeito 43
Figura 14 – Terreno 20x20x20 44
Figura 15 – Terreno 20x20x20 refeito 44
Figura 16 – Terreno 20x20x20 com textura padrão 45
Figura 17 – Terreno 40x15x40 com textura padrão 45
Figura 18 – Terreno 40x15x40, visão de cima 46
Figura 19 – Terreno 80x15x80 46
Figura 20 – Terreno 80x15x80, visão de cima 47

LISTA DE LISTAGENS

Listagem 1 – Função de criação do array 3D 18
Listagem 2 – Função que pinta veios de minérios 18
Listagem 3 – Função que gera montanhas 19
Listagem 4 – Função que escava cavernas 21
Listagem 5 – Função que arredonda o mapa 21
Listagem 6 – Função que adiciona camadas de rocha 22
Listagem 7 – Função que cria o cubo de ruído 23
Listagem 8 – Função que gera ruído de Perlin 24
Listagem 9 – Função que cria a caminhada 25
Listagem 10 – Função que pinta as esferas de blocos 28
Listagem 11 – Função que aumenta a resolução dos ruídos 29
Listagem 12 – Código completo do simulador 39

LISTA DE TERMOS E ABREVIAÇÕES

Screenshot – Captura de tela

HUD – Head-Up Display, informações que ficam na tela do jogo

Chunk – Pedaço do mapa do jogo

Engine – Motor gráfico para criação de jogos

Open Source – Código aberto

Pixel – Unidade digital, quadrado

Pixelart – Pintura digital feita com pixels

Grid - Grade

VFX – Efeitos visuais

SUMÁRIO

1. INTRODUÇÃO 14
1.1 Apresentação do tema 14
1.2 Geração Procedural 14
1.3 Justificativa 15
1.4 Objetivos Gerais 15
1.5 Objetivos Específicos 15
1.6 Cenário atual de jogos/simuladores 15
2. EXPLICAÇÃO DO FUNCIONAMENTO DOS ANTECESSORES 17
2.1 Infiniminer 17
2.1.1 Função que cria o array 3D 18
2.1.2 Função que "pinta" os veios de minérios no array 3D 18
2.1.3 Função que gera as montanhas do mundo 19
2.1.4 Função que "escava" as cavernas no mundo 21
2.1.5 Função que arredonda o mapa 21
2.1.6 Função que adiciona camadas de rocha no mapa 22
2.1.7 Função que cria um cubo com ruído com valores aleatórios 23
2.1.8 Função que gera o ruído de Perlin 24
2.1.9 Função que cria a "caminhada" pelo array 3D 25
2.1.10 Função que pinta as esferas de blocos 28
2.1.11 Função que aumenta a resolução dos ruídos gerados 29
2.2 Minecraft 31
2.2.1 Primeira Etapa – Mapa 2D 33
2.2.2 Segunda Etapa – Biomas 34
2.2.3 Terceira Etapa – Terreno 35
2.2.4 Quarta Etapa – Cavernas e ravinas 35
2.2.5 Quinta Etapa – Estruturas 35
2.2.6 Sexta Etapa – Decorações 36
2.2.7 Otimização 36
2.2.7.1 Chunks 36
2.2.7.2 Face Culling 37

SUMÁRIO

3. CÓDIGO DO SIMULADOR 39
3.1 Explicação dos algoritmos 41
4. FUNCIONAMENTO E EXECUÇÃO DOS TESTES 43
5. CONCLUSÕES E DESENVOLVIMENTOS FUTUROS 48
6. REFERÊNCIAS 49

1. INTRODUÇÃO

1.1. Apresentação do tema

Minecraft é um jogo mundialmente conhecido, sendo o jogo mais vendido de todos os tempos. Sua estrutura é simples, você precisa sobreviver em um mundo feito de blocos onde você coleta recursos e vai aprimorando sua base e seu personagem com ferramentas melhores, armaduras etc. Mas uma coisa que poucos sabem é que Minecraft foi inspirado, inicialmente sendo uma cópia, de um outro jogo chamado Infiminer, com uma estrutura muito parecida. Um mundo feito de blocos gerado de forma procedural onde você precisa coletar minérios e construir bases. Ambos os jogos são muito parecidos em sua estrutura, principalmente nos primórdios do Minecraft, quando o jogo ainda estava em um desenvolvimento inicial e não tinha tantas funções. Para o seguinte trabalho, vamos entender como esses jogos tratavam a geração procedural de um mundo e fazer a nossa própria versão em python utilizando a engine Ursina, criando um simulador de geração procedural de terrenos com blocos.

1.2. Geração Procedural

A geração procedural é uma técnica computacional voltada para a criação de conteúdos de forma automatizada, utilizando algoritmos, modelos matemáticos e regras definidas para produzir resultados variados. Embora hoje esteja fortemente associada ao desenvolvimento de jogos, sua origem é anterior a essas áreas, começando com modelos matemáticos. O princípio fundamental da geração procedural sempre foi permitir a construção de estruturas complexas a partir de instruções simples. Uma das primeiras aparições em jogos aconteceu em 1980 com o jogo “Rogue” que é considerado como o pai de um gênero inteiro de jogos, chamados “Rogue Like”(Kenny, 2021).

Na década de 1980, avanços como ruído de Perlin, desenvolvido por Ken Perlin, consolidaram o uso de algoritmos procedurais na indústria cinematográfica e na computação gráfica para criação de efeitos como fumaça, movimento da água e tecidos (BLATZ; KORN, 2017). Esse ruído gradiente permitiu a criação de texturas orgânicas e ambientes tridimensionais mais realistas, contribuindo diretamente para filmes icônicos e influenciando pesquisas futuras em modelagem e simulação. Com o avanço da capacidade computacional nas décadas seguintes, a geração procedural expandiu-se para diferentes áreas. No campo dos jogos digitais, consolidou-se como ferramenta para criação de mundos extensos e dinâmicos.

Porém, seu uso vai muito além desse contexto, abrangendo setores como urbanismo, arquitetura, música, animação, cinema e ciência de dados.

1.3. Justificativa

A geração procedural é uma tecnologia fundamental e que está presente em diversos campos, sua relevância está na capacidade de automatizar a criação de conteúdos e formas complexas e variadas, otimizando processos em larga escala. A importância da geração procedural é notória em áreas como cinema e VFX, na composição de cenários e efeitos, na arquitetura, na saúde e, onde é mais reconhecida, nos jogos. Assim, a geração procedural não é apenas um conceito teórico, mas um pilar que sustenta inovações práticas e impacta diretamente a forma como interagimos com ambientes digitais. Este projeto se justifica pela necessidade de compreender e analisar os princípios técnicos e as implicações da geração procedural, visando entender como essa tecnologia opera e seus limites, focando em um contexto de jogos eletrônicos, e mostrando uma aplicação com a criação de um simulador. Portanto, esse trabalho contribui para o avanço do conhecimento ao oferecer uma análise sobre geração procedural, solidificando a base teórica para futuros desenvolvimentos e aplicações em um cenário cada vez mais dependente de conteúdos gerados por algoritmos

1.4.Objetivos gerais

Criar um simulador de geração de mundos 3D com blocos utilizando a linguagem python com a engine ursina.

1.5 Objetivos específicos

- Construir um simulador capaz de gerar terrenos 3D feitos com blocos;
- Aplicar cores e texturas diferentes para cada camada do terreno;
- Implementar a geração de ruído com OpenSimplex;
- Criar terrenos aleatórios, mas que façam sentido, e que dependam de uma seed de geração;
- Implementar controles de jogo para o usuário interagir com o terreno gerado

1.6 Cenário atual de jogos/simuladores

Há diversos jogos que utilizam geração de mundo com ruído, como Lego Worlds, Dragon Quest Builders e Terraria (porém em 2D), mas para nossa pesquisa vamos focar no Infiminer

e no Minecraft. O Infiniminer foi lançado em 2009, mas logo descontinuado, já o Minecraft teve sua primeira versão oficial lançada em 2011, mas até hoje recebe atualizações, e mesmo assim sua estrutura base de geração de mundo não foi alterada, apenas refinada e melhorada. Ambos utilizam um tipo de ruído que define como certas características do mundo serão, como, por exemplo, onde o ruído é maior, teremos uma presença maior de montanhas, ou onde o ruído é menor teremos veios de minérios diferentes.

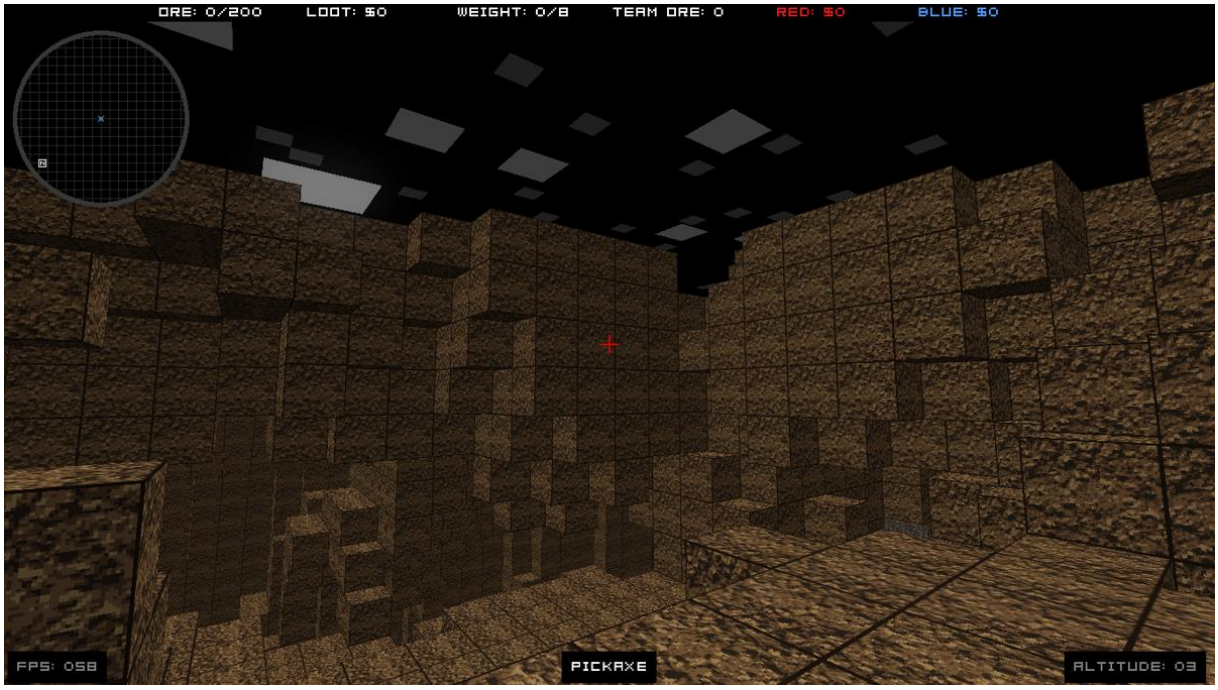
2. EXPLICAÇÃO DO FUNCIONAMENTO DOS ANTECESSORES

Para criação do código, primeiro serão analisados os algoritmos utilizados pelos antecessores Infiminer e Minecraft.

2.1. Infiniminer

Abaixo temos uma screenshot do jogo, mostrando como é a HUD e um pouco do terreno feito de blocos.

Figura 1: Screenshot do Infiminer



Fonte: MobyGames (2009)

Por ser Open Source, analisar como a geração de mundo do Infiminer funciona é uma tarefa mais simples. Ele foi feito em C#, utilizando o XNA Game Studio 3.0 da Microsoft, um ambiente de desenvolvimento que era voltado par criação de jogos. O código completo do jogo foi feito por Zach Barth (2009) e está disponível de forma Open Source no GitHub: https://github.com/craftworkgames/infiniminer. Para análise dos algoritmos que formam o mundo do jogo, é necessário acessar o arquivo CaveGenerator.cs, responsável pela maior parte do código de funcionamento do jogo, que fica no caminho infiminer/code/InfiminerServer. Abaixo, a explicação de como cada função funciona:

2.1.1 Função que cria o array 3D

Listagem 1: Função de criação do array 3D

```
BlockType[, ,] caveData = CaveGenerator.GenerateConstant(size,
BlockType.Dirt);
```

Fonte: Zach Barth (2009)

A função inicial do código, cria o array 3D com blocos do tipo dirt (terra). O Infiminer trabalha com diferentes blocos e com diferentes propriedades, sendo a maior parte do mundo constituída por blocos do tipo terra.

2.1.2 Função que "pinta" os veios de minérios no array 3D

Listagem 2: Função que pinta veios de minérios

```
float[, ,] oreNoise = CaveGenerator.GeneratePerlinNoise(32);
oreNoise = InterpolateData(ref oreNoise, 32, size);
for (int i = 0; i < oreFactor; i++)
CaveGenerator.PaintWithRandomWalk(ref caveData, ref oreNoise, size,
1, BlockType.Ore, false);
```

Fonte: Zach Barth (2009)

Essa função gera um volume 3D de ruído de Perlin, que é feito em baixa resolução, então faz um InterpolateData para aumentar esse ruído para a resolução final. Faz um for com a função PaintWithRandomWalk que caminha de forma aleatória pelo array e "pinta" veios de minérios usando o mapa de ruído gerado.

2.1.3 Função que gera as montanhas do mundo

Listagem 3: Função que gera montanhas

```
            float[, ,] mountainNoise =
CaveGenerator.GeneratePerlinNoise(32);

            mountainNoise = InterpolateData(ref mountainNoise, 32,
size);

            for (int x = 0; x < size; x++)

                for (int y = 0; y < size; y++)

                    for (int z = 0; z <= Defines.GROUND_LEVEL * 2;
z++)

                        mountainNoise[x, y, z] = z < 3 ? 0 :
Math.Min(1, z / (Defines.GROUND_LEVEL * 2));

            float[, ,] gradient =
CaveGenerator.GenerateGradient(size);

            CaveGenerator.AddDataTo(ref mountainNoise, ref gradient,
size, 0.1f, 0.9f);

            BlockType[, ,] mountainData =
CaveGenerator.GenerateConstant(size, BlockType.None);

            int numMountains = randGen.Next(size, size * 3);

            for (int i = 0; i < numMountains; i++)

                CaveGenerator.PaintWithRandomWalk(ref mountainData,
ref mountainNoise, size, randGen.Next(2, 3), BlockType.Dirt, false);

            for (int x = 0; x < size; x++)

                for (int y = 0; y < size; y++)

                    for (int z = 0; z <= Defines.GROUND_LEVEL; z++)

                        if (mountainData[x, y, z] == BlockType.None)

                            caveData[x, y, z] = BlockType.None;
```

Fonte: Zach Barth (2009)

O bloco de código acima é responsável pela geração das montanhas da superfície do mundo. Ele começa criando mountainNoise com ruído de Perlin em baixa resolução que é interpolado, ou seja, aumentado para a resolução necessária. Ele pega todos os blocos que estejam em até 2x o nível do solo (o nível do solo é fixo no código como 8) e substitui o valor do ruído para que dependam apenas da altura z, isso garante que as montanhas se formem a partir das regiões mais altas do mundo. É feito um gradiente vertical que é combinado com o mountainNoise, isso é feito para dar um aspecto mais natural no terreno e com mais

variabilidade. Ele, então, prepara um array 3D preenchido com blocos vazios que será preenchido pelas montanhas geradas. O numMountains é um número aléatório que determina quantas “caminhadas” serão feitas para formação das montanhas, para cada PaintWithRandomWalk feito ele seleciona uma posição aleatória no array 3D, caminha por ele e a cada passo faz uma esfera de raio 2 com blocos do tipo dirt. Essas esferas vão se aglomerando e criando montanhas. Por serem esferas, os topos das montanhas são sempre achatados, eles não ficam pontudos, como é possível ver na imagem abaixo:

Figura 2: Screenshot mostrando o terreno do Infiminer

Fonte: MobyGames (2009)

Ele, então, Percorre todo o array 3D do mundo (caveData) acima do nível do solo (GROUND_LEVEL), e compara com mountainData para ver onde serão os blocos de dirt das montanhas e onde serão os blocos vazios do céu.

2.1.4 Função que “escava” as cavernas no mundo

Listagem 4: Função que escava cavernas

```
            float[, ,] caveNoise =
CaveGenerator.GeneratePerlinNoise(32);

            caveNoise = InterpolateData(ref caveNoise, 32, size);

            gradient = CaveGenerator.GenerateGradient(size);

            CaveGenerator.AddDataTo(ref caveNoise, ref gradient,
size, 1 - gradientStrength, gradientStrength);

            int cavesToCarve = randGen.Next(size / 8, size / 2);

            for (int i = 0; i < cavesToCarve; i++)

                CaveGenerator.PaintWithRandomWalk(ref caveData, ref
caveNoise, size, randGen.Next(1, 2), BlockType.None, false);
```

Fonte: Zach Barth (2009)

Começa gerando ruído de Perlin em baixa resolução que é interpolado. Então, cria um gradiente vertical que dá preferência para cavernas em certas profundidades. Mistura o caveNoise com o gradiente e define quantas “caminhadas” serão feitas para escavar as cavernas. Faz um loop que, para cada iteração, realiza as caminhadas pelo mapa 3D substituindo os blocos já existentes por blocos de ar para criar as cavernas. Caso chegue no limite do mapa, ele para de escavar.

2.1.5 Função que arredonda o mapa

Listagem 5: Função que arredonda o mapa

```
            float[, ,] sphereGradient =
CaveGenerator.GenerateRadialGradient(size);

            cavesToCarve = randGen.Next(size / 8, size / 2);

            for (int i = 0; i < cavesToCarve; i++)

                CaveGenerator.PaintWithRandomWalk(ref caveData, ref
sphereGradient, size, randGen.Next(1, 2), BlockType.None, true);
```

Fonte: Zach Barth (2009)

O código acima começa criando um mapa gradiente esférico que começa do centro com valores que vão crescendo conforme se aproximam das bordas do mundo. É selecionado um

número aleatório de caminhadas e faz um loop que não para nas bordas, ao chegar no limite do mapa ele continua e vai “aparando” e deixando esférico.

2.1.6 Função que adiciona camadas de rocha no mapa

Listagem 6: Função que adiciona camadas de rocha

```
        public static void AddRocks(ref BlockType[, ,] data, int
size)
        {
            int numRocks = randGen.Next(size, 2*size);
            CaveInfo += " numRocks=" + numRocks;
            for (int i = 0; i < numRocks; i++)
            {
                int x = randGen.Next(0, size);
                int y = randGen.Next(0, size);
                float zf = 0;
                for (int j = 0; j < 4; j++)
                    zf += (float)randGen.NextDouble();
                zf /= 2;
                zf = 1 - Math.Abs(zf - 1);
                int z = (int)(zf * size);
                int rockSize = (int)((randGen.NextDouble() +
randGen.NextDouble() + randGen.NextDouble() + randGen.NextDouble())
/ 4 * 8);
                PaintAtPoint(ref data, x, y, z, size, rockSize,
BlockType.Rock);
            }
        }
```

Fonte: Zach Barth (2009)

Começa escolhendo entre size e 2 x size para quantas rochas gerar, quanto maior o mundo, mais blocos de pedra. A distribuição desses blocos é empurrada para camadas mais baixas e então “pinta” esferas de pedra com raios do tamanho de rockSize, gerando as camadas de pedra no subsolo do mundo.

2.1.7 Função que cria um cubo com ruído com valores aleatórios

Listagem 7: Função que cria o cubo de ruído

```
        // Generates a cube of noise with sides of length size.
Noise falls in a linear

        // distribution ranging from 0 to magnitude.

        public static float[, ,] GenerateNoise(int size, float
magnitude)

        {

            float[,,] noiseArray = new float[size, size, size];

            for (int x = 0; x < size; x++)

                for (int y = 0; y < size; y++)

                    for (int z = 0; z < size; z++)

                        noiseArray[x, y, z] =
(float)randGen.NextDouble() * magnitude;

            return noiseArray;

        }
```

Fonte: Zach Barth (2009)

A função acima cria um array 3D e o preenche com valores aleatórios entre 0 e magnitude (64) de forma uniforme. Serve para criar a variação no terreno ao ser combinado com outras funções como a de ruído de Perlin.

2.1.8 Função que gera o ruído de Perlin

Listagem 8: Função que gera ruído de Perlin

```
// Generates some perlin noise!
public static float[,,] GeneratePerlinNoise(int size)
{
    float[,,] data = new float[size, size, size];
    float[,,] noise = null;
    for (int f = 4; f < 32; f *= 2)
    {
        noise = GenerateNoise(f, 2f / f);
        noise = InterpolateData(ref noise, f, size);
        AddDataTo(ref data, ref noise, size);
    }

    return data;
}
```

Fonte: Zach Barth (2009)

Faz um array 3D com um loop onde gera grids de ruído com diferentes frequências. Expande a grid com InterpolateData para o tamanho (size) do mundo e soma no data com AddDataTo, contribuindo para as camadas de ruído que criam os padrões do mundo.

2.1.9 Função que cria a “caminhada” pelo array 3D

Listagem 9: Função que cria a “caminhada”

```
        public static void PaintWithRandomWalk(ref BlockType[, ,]
caveData, ref float[, ,] noiseData, int size, int paintRadius,
BlockType paintValue, bool dontStopAtEdge)
        {
            int x = randGen.Next(0, size);
            int y = randGen.Next(0, size);
            int z = randGen.Next(0, size);
            if (z < size/50)
                z = 0;
            int count = 0;
            while (dontStopAtEdge == false || count < size)
            {
                float oldNoise = noiseData[x, y, z];
                PaintAtPoint(ref caveData, x, y, z, size,
paintRadius+1, paintValue);
                int dx = randGen.Next(0, paintRadius * 2 + 1) -
paintRadius;
                int dy = randGen.Next(0, paintRadius * 2 + 1) -
paintRadius;
                int dz = randGen.Next(0, paintRadius * 2 + 1) -
paintRadius;
                x += dx;
                y += dy;
                z += dz;
                if (x < 0 || y < 0 || x >= size || y >= size || z >=
size)
                {
                    if (dontStopAtEdge)
                    {
                        count += 1;
                        if (x < 0) x = 0;
                        if (y < 0) y = 0;
                        if (z < 0) z = 0;
```

```
                        if (x >= size) x = size - 1;
                        if (y >= size) y = size - 1;
                        if (z >= size) z = size - 1;
                    }
                    else
                        break;
                }
                if (z < 0)
                    z = 0;
                float newNoise = noiseData[x, y, z];
                // If we're jumping to a higher value on the noise
gradient, move twice as far.
                if (newNoise > oldNoise)
                {
                    PaintAtPoint(ref caveData, x, y, z, size,
paintRadius+1, paintValue);
                    x += dx;
                    y += dy;
                    z += dz;
                    if (x < 0 || y < 0 || x >= size || y >= size ||
z >= size)
                    {
                        if (dontStopAtEdge)
                        {
                            count += 1;
                            if (x < 0) x = 0;
                            if (y < 0) y = 0;
                            if (z < 0) z = 0;
                            if (x >= size) x = size - 1;
                            if (y >= size) y = size - 1;
                            if (z >= size) z = size - 1;
                        }
                        else
                            break;
```

```
                    }
                    if (z < 0)
                        z = 0;
                }
            }
        }
```

Fonte: Zach Barth (2009)

A função acima seleciona um ponto aleatório no array 3D (x,y,z) e se for muito próximo do fundo, força o z para 0. Faz um loop para cada caso, dontStopAtEdge == false que sai do loop caso saia dos limites do mundo, e dontStopAtEdge == true que vai continuar rodando até a variável count, que registra o número de colisões fora dos limites do mapa, atingir o valor da variável size. Em cada passo dessa "caminhada" ele registra o valor do mapa de ruído do bloco em que está, chama a função PaintAtPoint para pintar uma esfera no ponto atual, escolhe uma direção aleatória (dx,dy,dz) e verifica se saiu das bordas do array, ativando a função correspondente em cada caso. Por fim, compara o newNoise com oldNoise, os valores de ruído do bloco atual com o próximo, e caso o novo seja maior, pinta a esfera novamente e faz mais um passo, ou seja, áreas com ruídos maiores são preferidas pela função.

2.1.10 Função que pinta as esferas de blocos

Listagem 10: Função que pinta as esferas de blocos

```
        public static void PaintAtPoint(ref BlockType[, ,] caveData,
int x, int y, int z, int size, int paintRadius, BlockType
paintValue)

        {

            for (int dx = -paintRadius; dx <= paintRadius; dx++)

                for (int dy = -paintRadius; dy <= paintRadius; dy++)

                    for (int dz = -paintRadius; dz <= paintRadius;
dz++)

                        if (x+dx >= 0 && y+dy>= 0 && z+dz >= 0 &&
x+dx < size && y+dy < size && z+dz < size)

                            if
(dx*dx+dy*dy+dz*dz<paintRadius*paintRadius)

                                caveData[x + dx, y + dy, z + dz] =
paintValue;

        }
```

Fonte: Zach Barth (2009)

Essa função é responsável por pintar os cubos no array 3D com determinados tipos de blocos, e de raio 1, ou seja, 3 blocos de diâmetro. O formato fica parecido com uma cruz 3D.

2.1.11 Função que aumenta a resolução dos ruídos gerados

Listagem 11: Função que aumenta a resolução dos ruídos

```
        // Resizes dataIn, with size sizeIn, to be of size sizeOut.
        public static float[, ,] InterpolateData(ref float[, ,]
dataIn, int sizeIn, int sizeOut)
        {
            Debug.Assert(sizeOut > sizeIn, "sizeOut must be greater
than sizeIn");
            Debug.Assert(sizeOut % sizeIn == 0, "sizeOut must be a
multiple of sizeIn");
            float[,,] dataOut = new float[sizeOut, sizeOut,
sizeOut];
            int r = sizeOut / sizeIn;
            for (int x=0; x<sizeOut; x++)
                for (int y = 0; y < sizeOut; y++)
                    for (int z = 0; z < sizeOut; z++)
                    {
                        int xIn0 = x / r,       yIn0 = y / r,
zIn0 = z / r;
                        int xIn1 = xIn0 + 1,    yIn1 = yIn0 + 1,
zIn1 = zIn0 + 1;
                        if (xIn1 >= sizeIn)
                            xIn1 = 0;
                        if (yIn1 >= sizeIn)
                            yIn1 = 0;
                        if (zIn1 >= sizeIn)
                            zIn1 = 0;
                        float v000 = dataIn[xIn0, yIn0, zIn0];
                        float v100 = dataIn[xIn1, yIn0, zIn0];
                        float v010 = dataIn[xIn0, yIn1, zIn0];
                        float v110 = dataIn[xIn1, yIn1, zIn0];
                        float v001 = dataIn[xIn0, yIn0, zIn1];
                        float v101 = dataIn[xIn1, yIn0, zIn1];
                        float v011 = dataIn[xIn0, yIn1, zIn1];
```

```
                        float v111 = dataIn[xIn1, yIn1, zIn1];
                        float xS = ((float)(x % r)) / r;
                        float yS = ((float)(y % r)) / r;
                        float zS = ((float)(z % r)) / r;
                        dataOut[x, y, z] =  v000 * (1 - xS) * (1 -
yS) * (1 - zS) +
                                            v100 * xS * (1 - yS) *
(1 - zS) +
                                            v010 * (1 - xS) * yS *
(1 - zS) +
                                            v001 * (1 - xS) * (1 -
yS) * zS +
                                            v101 * xS * (1 - yS) *
zS +
                                            v011 * (1 - xS) * yS *
zS +
                                            v110 * xS * yS * (1 -
zS) +
                                            v111 * xS * yS * zS;
                    }
            return dataOut;
        }
```

Fonte: Zach Barth (2009)

Essa função pega o array gerado e faz com que cada ponto dele se “traduza” em um cubo X x Y x Z, resultando em um mapa de ruído mais suavizado. Ela também garante que o valor de saída seja maior do que o de entrada.

2.2. Minecraft

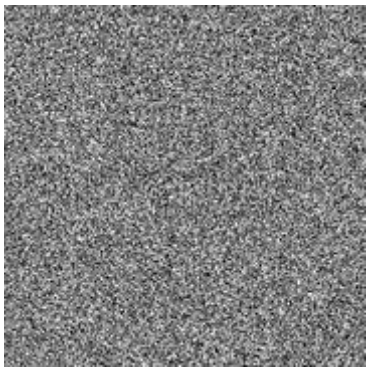
Abaixo, uma screenshot do jogo, mostrando alguns elementos do HUD e um pouco do terreno.

Figura 3: Screenshot do Minecraft

Fonte: Minecraft Wiki (2025)

O código do Minecraft, diferente do Infiminer, é fechado, portanto para entender como sua geração de mundo funciona, é necessário procurar por pesquisas já feitas sobre esse tema e investigar declarações do próprio criado do jogo, Marcus Persson, o “Notch”. O jogo foi feito inicialmente na linguagem Java e posteriormente teve uma versão em C++. Foi feito do zero, sem uma engine, apenas com a ajuda de um framework chamado LightWeight Java Game Library (LWJGL) (Game Developer, 2011). Segundo o site Minecraft Wiki (2025), e o documentário “How Minecraft is Actually Works”, de Alan Zucconi (2022), ruído de Perlin é utilizado em diferentes frequências e depois somado, dando uma aleatoriedade “fluída” para o mundo, que faça sentido. Nas imagens abaixo é possível comparar a diferença entre um ruído branco comum e o ruído gerado por uma função de Perlin. Como citado por Melnychuk Vladyslav (2020) em sua dissertação “Landscape generation using procedural generation techniques” o ruído de Perlin foi criado em 1983 por Ken Perlin para o filme Tron e, anos depois, em 2001, Ken criou o Simplex como um sucessor do ruído de Perlin, sendo mais otimizado, menos complexo e exigindo menos do computador

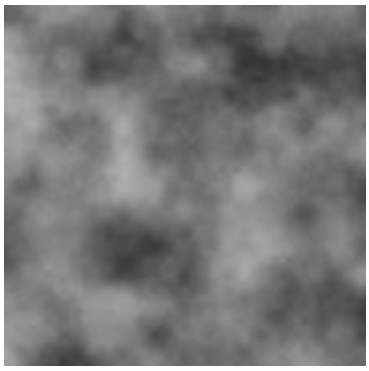
Figura 4: Ruído branco gerado por uma função aleatória



Fonte: Minecraft Wiki (2025)

O imagem abaixo é composta por múltiplas iterações de ruído de Perlin combinadas para chegar em um resultado mais suave e natural.

Figura 5: Ruído gerado por Perlin com múltiplas oitavas combinadas



Fonte: Minecraft Wiki (2025)

É possível ver que o ruído branco é parecido com estática, uma coisa desorganizada, enquanto o ruído gerado por Perlin se assemelha a uma fumaça, muito mais suave. Alan Zucconi (2022), em seu documentário aborda de forma muito didática como as diferentes etapas da geração do mundo do jogo funcionam. Minecraft Wiki (2025) também é utilizada como fonte de informação por ser um dos maiores repositórios relacionados a Minecraft. Primeiro, uma seed é gerada aleatoriamente ou definida pelo jogador, essa semente será utilizada para definir os padrões do mundo, para gerar os ruídos de Perlin e garantir que mundos feitos com a mesma semente sejam iguais.

2.2.1 Primeira Etapa – Mapa 2D

A primeira etapa na geração de mundo do Minecraft consiste em gerar um mapa 2D responsável por definir onde será água e onde será terra. Os pixels azuis representam água e os verdes representam terra.

Figura 6: Exemplo do mapa de água/terra gerado

Fonte: Autoria própria (2025)

Esse mapa gerado é então ampliado.

Figura 7: Exemplo do mapa ampliado

Fonte: Autoria própria (2025)

O mapa é ampliado diversas vezes até chegar no tamanho final desejado.

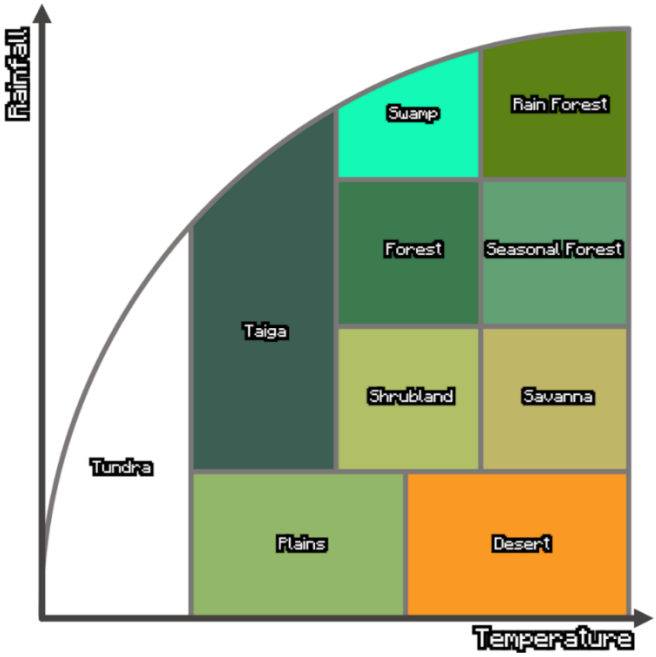
Figura 8: Exemplo do mapa ampliado finalizado

Fonte: Autoria própria (2025)

2.2.2 Segunda Etapa - Biomas

Após definição do mapa de terra, é feito uma junção de várias camadas de ruído para compor o mapa de biomas em cima do mapa já gerado. O código do jogo define cada bioma de acordo com a temperatura e chance de chuva. O jogo ainda tem uma função para garantir que haja uma suavização entre a transição de biomas, para que um deserto, que é seco e sem chuva, não fique perto de um pântano, por exemplo, que é mais ameno e tem uma grande incidência de chuva. A imagem abaixo feita por Alan Zucconi (2022) representa bem como isso é dividido.

Figura 9: Gráfico com os diferentes biomas do jogo



Fonte: Alan Zucconi (2022)

Os biomas do jogo definem, não somente as características deles, como tipo de bloco, animais que vão aparecer, vegetaçãoe etc, mas também a altura do eixo Y.

Figura 10: Exemplo da divisão dos biomas

Fonte: Autoria própria (2025)

2.2.3 Terceira Etapa – Terreno

Com o mapa de biomas pronto, gera o terreno 3D do jogo de acordo com as caracteríscias de altura e profundidade de cada bioma e de acordo com o mapa de ruído, que tem várias camadas sobrepostas, gerando um terreno mais natural. O terreno é gerado inteiramente de blocos de pedra que depois serão substituídos por blocos de neve em ambientes frios, blocos de areia no deserto, blocos de cascalho nos oceanos etc. É definido, também, que todo bloco abaixo de 62 se torne água, criando riachos.

2.2.4 Quarta Etapa – Cavernas e ravinas

Com o terreno 3D pronto, escava cavernas e ravinas no solo de forma parecida com o infiminer, com uma esfera que vai escavando o solo em coordenadas definidas pelo mapa de ruído, mas nesse caso é chamado de Perlin Worms, minhocas de Perlin.

2.2.5 Quinta Etapa – Estruturas

Adiciona estruturas no jogo como vilas, templos do deserto, masmorras, minas abandonadas etc. Essas estruturas dão ao jogo mais personalidade e opções de jogabilidade.

2.2.6 Sexta Etapa – Decorações

Adicionas árvores, flores, minérios e outras decorações que deixam o mundo mais vivo. No caso dos minérios, também utiliza funções muito similares ao Infiminer, como o número fixo de diamantes por região (chunk) e definidos em camadas mais baixas, os tornando mais raros de achar.

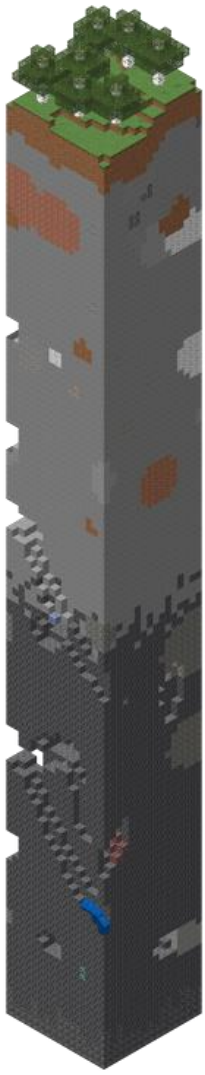
2.2.7 Otimização

O mapa do Minecraft tem uma área de 60 milhões por 60 milhões de blocos, com 384 blocos de altura. Com um mapa tão grande assim seria praticamente impossível rodar o jogo em qualquer máquina, por mais potente que seja. Por isso, o Minecraft possui diversas funções de otimização que possibilitam que o jogo seja jogado até mesmo em celulares.

2.2.7.1 Chunks

Segundo a Minecraft Wiki (2025) e Alan Zucconi (2022), o mapa do Minecraft é dividido em chunks com área horizontal de 16x16 e altura de 384 blocos. Esse chunks são uma das formas de otimização do jogo. Os chunks são gerados de acordo com o caminhar do jogador, onde chunks mais a frente são gerados conforme o jogador se aproxima e chunks mais para trás são “descarregados” liberando memória e não pesando o jogo. Quanto mais distante um chunk está, menos características ele tem, como, por exemplo, em chunks muito distantes não é possível ver a presença de animais ou monstros, ou o funcionamento de máquinas como fornalhas deixa de acontecer. A quantidade de chunks que o jogador vê pode ser alterada nas configurações do jogo.

Figura 11: Representação de um chunk do minecraft



Fonte: Minecraft Wiki (2025)

2.2.7.2 Face Culling

Face Culling é uma técnica que esconde as faces dos objetivos que não estão voltados para o jogador, ou seja, que o jogador não consegue ver. Isso é aplicado no Minecraft de modo que mesmo que você esteja em um chunk, os blocos que estejam 2 níveis abaixo de você, por exemplo, existem, mas como não estão visíveis para o jogador, não são 100% carregados, “aparecendo” apenas quando quebramos os blocos que estão na frente ou acima. O desenvolvedor da Mojang, Tommaso Checchi (2014) descreveu em seu blog, na época que estavam trabalhando na versão de celular do Minecraft, os desafios e a solução encontrada para tornar as cavernas do jogo mais otimizadas para dispositivos móveis. Ele criou um

método chamado de Cave Culling que esconde as cavernas que o jogador não está vendo diretamente.

3. CÓDIGO DO SIMULADOR

Abaixo, o código completo comentado do simulador.

Listagem 12: Código completo do simulador

```
from ursina import *
from ursina.prefabs.first_person_controller import
FirstPersonController
from ursina import held_keys
import random
from opensimplex import OpenSimplex

app = Ursina() # Cria a instância do ursina

wdb = False

window.exit_button.visible = wdb
window.fps_counter.enabled = wdb
window.collider_counter.enabled = wdb
window.entity_counter.enabled = wdb

world_scale = 15
world_height = 15

window.color = color.rgb(66/255, 227/255, 247/255)

class Block(Entity):
    def __init__(self, position=(0,0,0), color=color.white,
texture='white_cube'):
        super().__init__(
            position = position,
            model = 'cube',
            color = color,
            texture = texture,
            scale = 1,
            collider = 'box'
        )
        self._orig_style = (self.color, self.texture)

seed = random.randint(0, 99999)
noise_scale = 0.15
noise = OpenSimplex(seed=seed)

def column_height(x, z, scale=noise_scale, max_h = world_height):
    n = noise.noise2(x * scale, z * scale)
    n = (n + 1.0) / 2.0
    h = int(n * (max_h - 1)) + 1
    return h

def style_height(y, max_h=world_height):
    if y < max_h * 0.2:
        return (color.rgb(50/255,169/255,201/255), 'agua.png')
    elif y < max_h * 0.6:
```

```
        return (color.rgb(145/255,60/255,32/255), 'terra.png')
    else:
        return (color.rgb(22/255,191/255,38/255), 'grama.png')

print(f'Gerando mundo com a seed {seed}')
for x in range(world_scale):
    for z in range(world_scale):
        h = column_height(x, z)
        for y in range(h):
            cor, tex = style_height(y)
            Block(position=(x, y, z), color=cor, texture=tex)

player = FirstPersonController()
player.cursor.visible = True
player.gravity = 0.0
player.speed = 3
player.jump_height = 1

player.x = world_scale / 2
player.y = world_height + 2
player.z = world_scale / 2

hovered_last = None

def highlight_hovered():
    global hovered_last
    e = mouse.hovered_entity
    if e is hovered_last:
        return
    if hovered_last and isinstance(hovered_last, Block):
        try:
            orig_col, orig_tex = hovered_last._orig_style
            hovered_last.color = orig_col
            hovered_last.texture = orig_tex
        except Exception:
            pass
    hovered_last = None

    if e and isinstance(e, Block):
        hovered_last = e
        try:
            e.color = e.color.tint(0.3)
        except Exception:
            pass

def input(key):
    if key == 'escape':
        application.quit()

    if key == 'v':
        if player.gravity == 0.0:
            player.gravity = 0.3
        else:
```

```
            player.gravity = 0.0

    if key == 'f':
        player.x = world_scale / 2
        player.y = world_height + 2
        player.z = world_scale / 2

    if key == 'r':
        for e in list(scene.entities):
            if isinstance(e, Block):
                destroy(e)

        global seed, noise
        seed = random.randint(0, 99999)
        noise = OpenSimplex(seed=seed)
        for x in range(world_scale):
            for z in range(world_scale):
                h = column_height(x, z)
                for y in range(h):
                    col, tex = style_height(y)
                    Block(position=(x, y, z), color=col,
texture=tex)
        print('Nova seed: ', seed)

    if key == 'left mouse down' and mouse.hovered_entity:
        destroy(mouse.hovered_entity)

def update():
    highlight_hovered()
    if player.gravity == 0.0:
        if held_keys['space']:
            player.y += 0.5
        if held_keys['shift']:
            player.y -= 0.5

app.run()
```

Fonte: Autoria própria (2025)

3.1 Explicação dos algoritmos

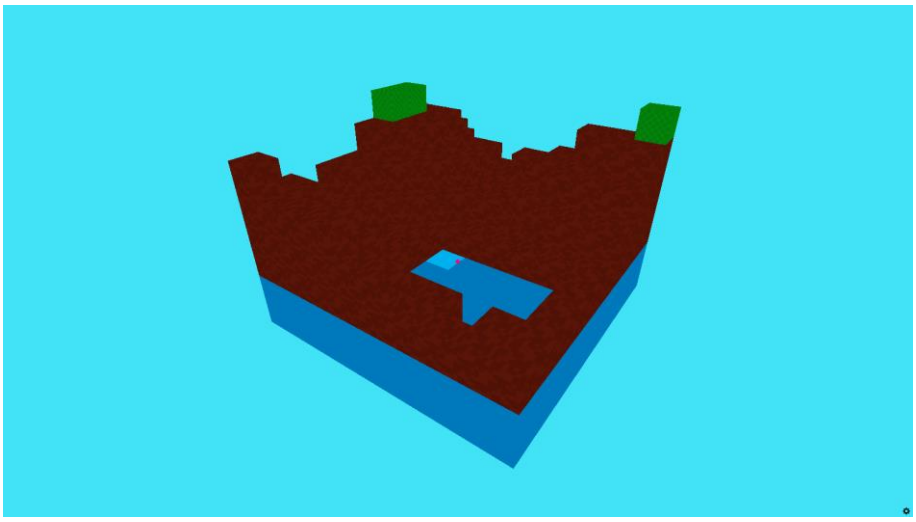
Para criação do terreno do simulador, foi utilizado OpenSimplex, por ser mais fácil de implementar e mais eficiente, como implementado por Vladyslav Melnychuk (2020). Uma seed aleatória é escolhida entre 0 e 99999, essa semente é utilizada na função do OpenSimplex para gerar o ruído que será utilizado para determinar o tamanho de cada coluna do mundo (eixo Y). Para criação do código utilizando a engina Ursina (2020), a documentação no site foi de grande ajuda pelos exemplos e tutoriais já feitos. O Ursina já tem objetos prontos como esferas e blocos, então a classe Block é iniciada com as coordenadas, textura e cor padrões. O tamanho do terreno é definido no código, e no exemplo acima é fixo

em 15x15x15, então para cada coordenada dentro dessa área, e seguindo o mapa de ruído gerado, um bloco é colocado, gerando assim o terreno do simulador. 3 tipos de bloco foram definidos: terra, grama e água. Caso o bloco esteja em até 20% da altura do mundo, será do tipo água, com cor e textura próprios, caso o bloco esteja entre 20% e 60%, será do tipo terra, e acima disso será grama. Também foram implementadas funções de controle de primeira pessoa, para o jogado interagir com o terreno andando por ele, pulando e quebrando blocos e uma função de voo ao apertar o botãoV. Por último, sempre que o jogador apertar a tecla R, um novo terreno, com uma nova seed, será gerado, mantendo os padrões de tamanho definidos.

4. FUNCIONAMENTO E EXECUÇÃO DOS TESTES

É possível interagir com o terreno desativando o modo de voo clicando no V e caminhar por ele utilizando WASD, pulando com a barra de espaço e quebrando blocos com o botão esquerdo do mouse. Primeiro, criando um terreno básico de 10 de altura, 10 de largura e 10 de comprimento:

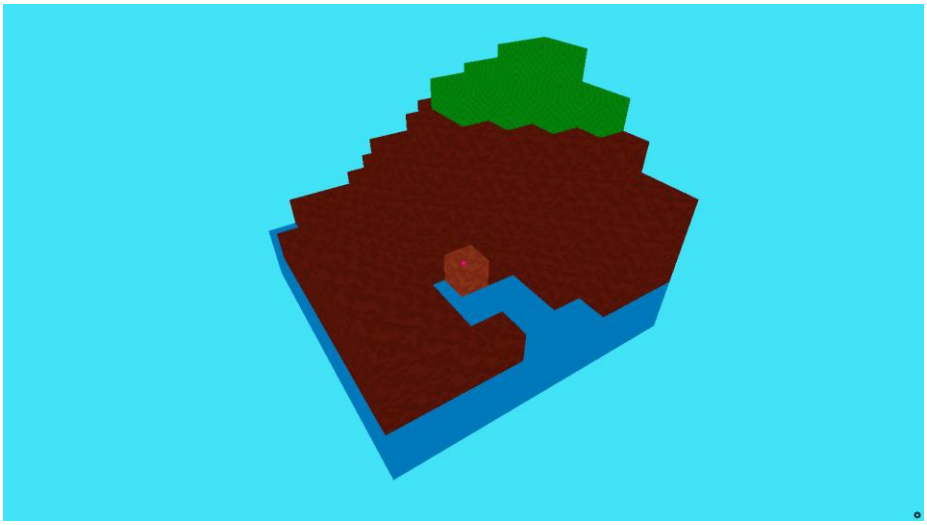
Figura 12 – Terreno 10x10x10



Fonte: Autoria própria

Apertando R para gerar um novo terreno:

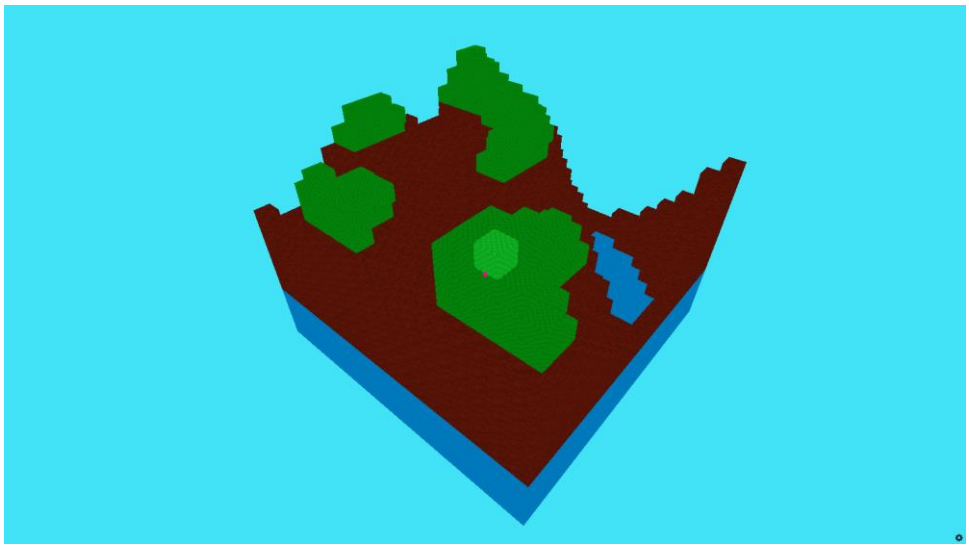
Figura 13 – Terreno 10x10x10 refeito



Fonte: Autoria própria

Alterando as dimensões do terreno no código para 20x20 e gerando um novo mapa:

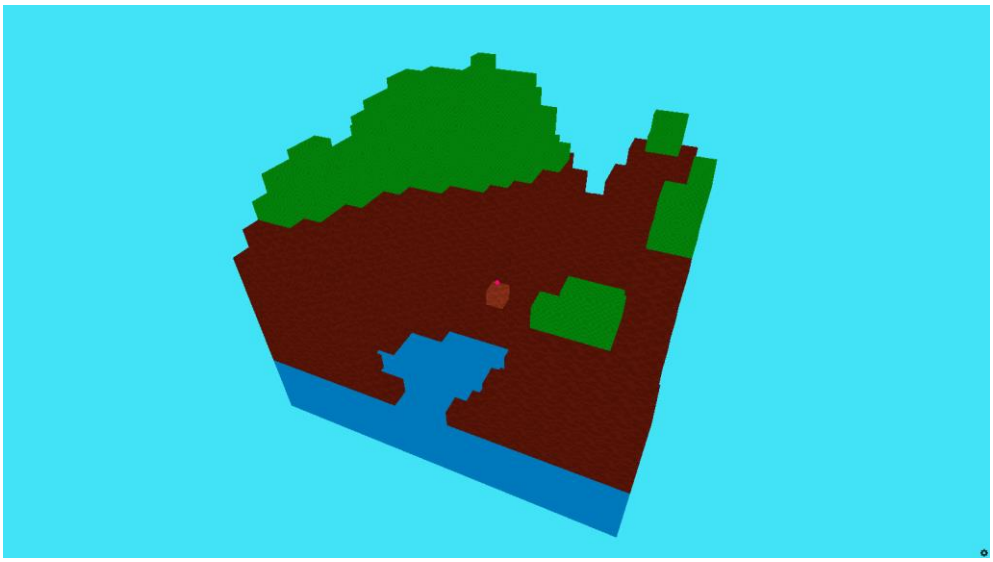
Figura 14 – Terreno 20x20x20



Fonte: Autoria própria

Apertando R e gerando o terreno novamente:

Figura 15 – Terreno 20x20x20 refeito

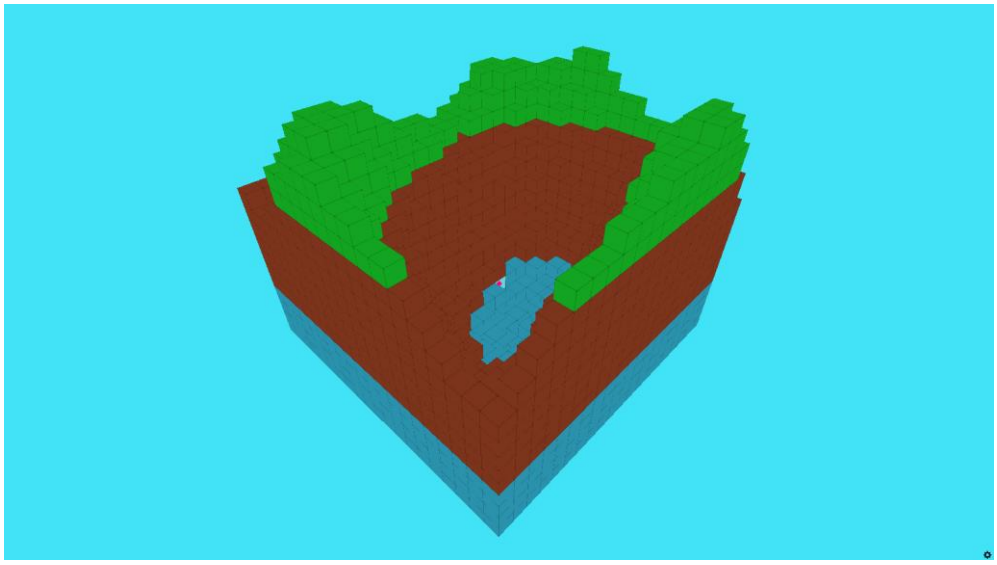


Fonte: Autoria própria

A textura utilizada anteriormente foi criada em pixelart com faces de 8x8, utilizaremos uma textura padrão do Ursina para demonstrar melhor a separação dos blocos.

Apertando R novamente e utilizando uma textura diferente dessa vez, onde é possível ver melhor os blocos individuais.:

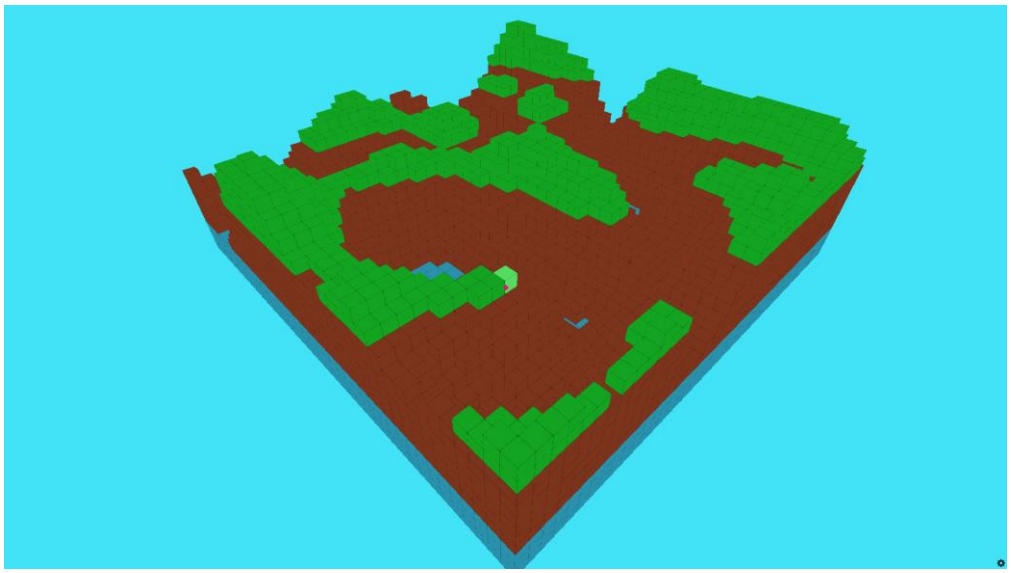
Figura 16 – Terreno 20x20x20 com textura padrão



Fonte: Autoria própria

Alterando as dimensões do mapa para 40x15, sendo extenso, mas com montanhas pequenas:

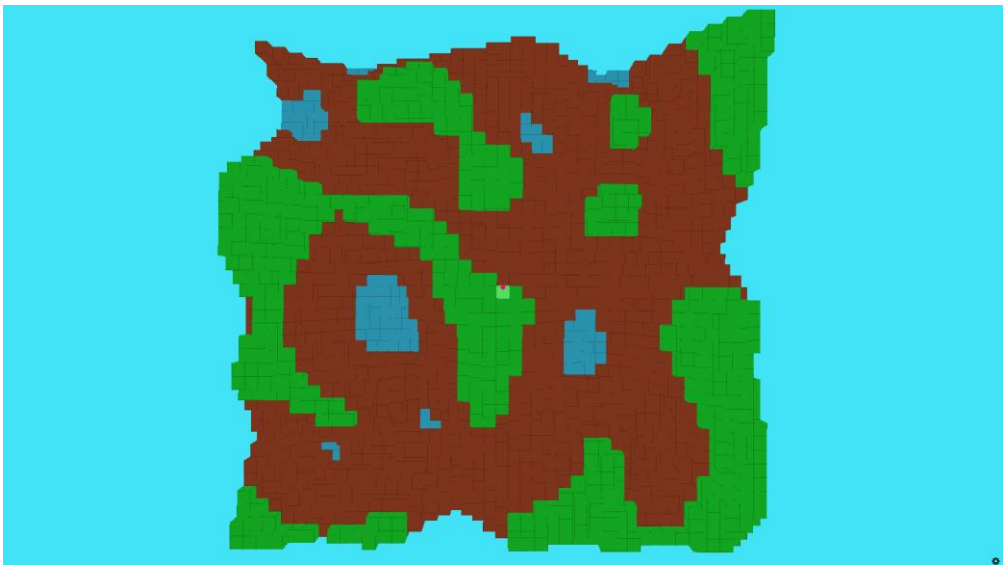
Figura 17 – Terreno 40x15x40 com textura padrão



Fonte: Autoria própria

Visão de cima do mundo:

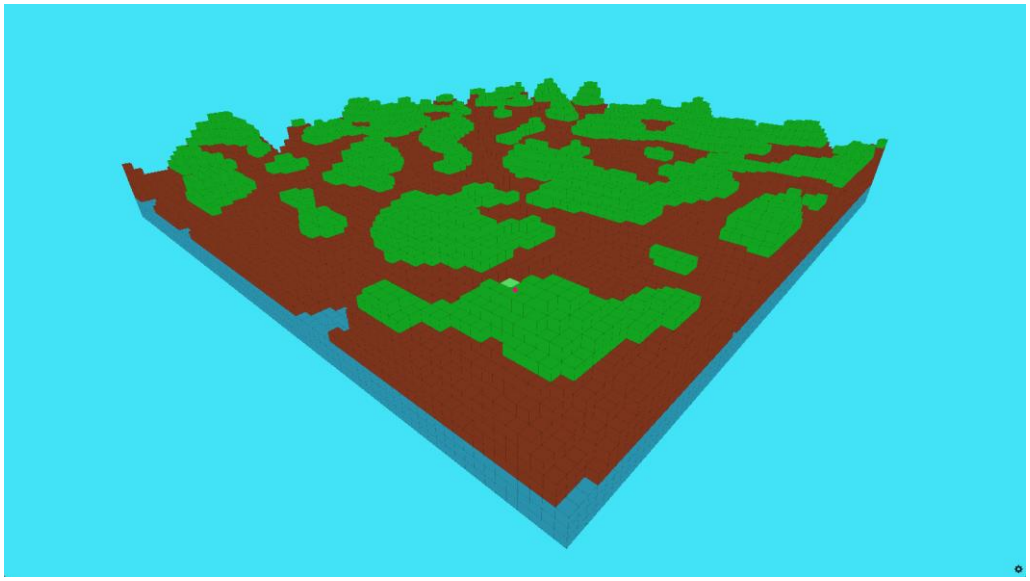
Figura 18 – Terreno 40x15x40, visão de cima



Fonte: Autoria própria

Criando um terreno bem maior:

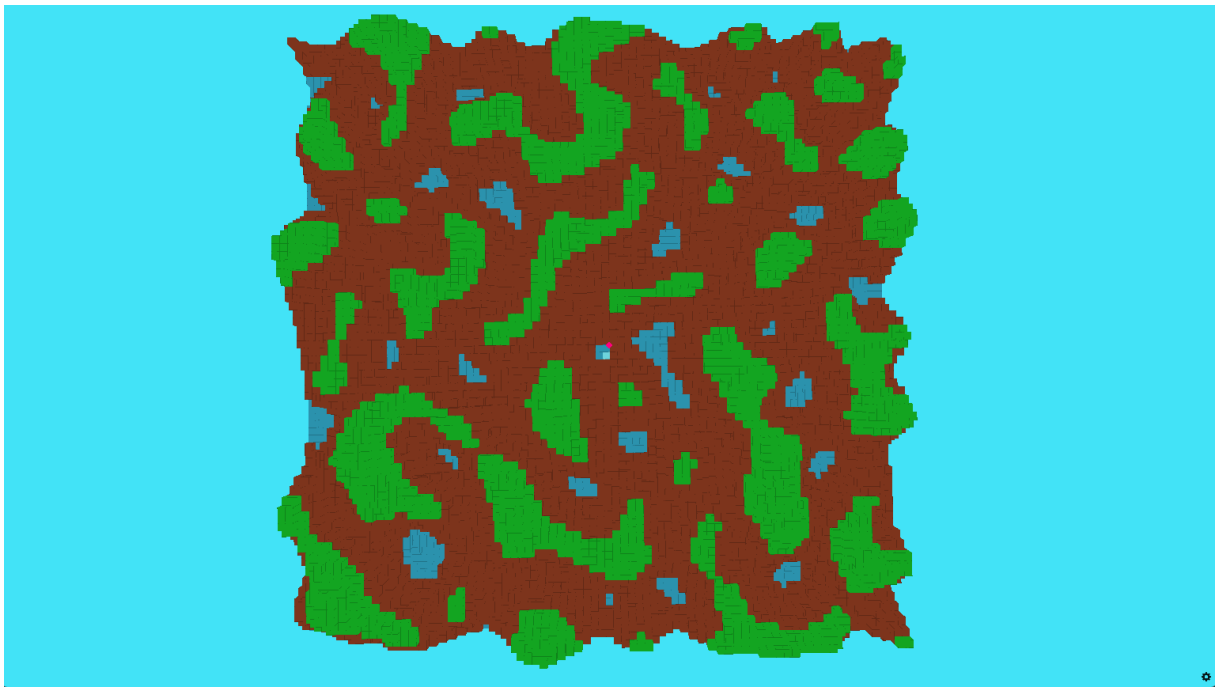
Figura 19 – Terreno 80x15x80



Fonte: Autoria própria

Visão de cima do terreno da imagem acima:

Figura 20 – Terreno 80x15x80, visão de cima



Fonte: Autoria própria

O tempo de carregamento do terreno depende da capacidade do computador e das dimensões do mundo que foram definidas.

5. CONCLUSÕES E DESENVOLVIMENTOS FUTUROS

Com o trabalho e a pesquisa pudemos entender melhor como os jogos que parecem ser tão simples funcionam de formas complexas que não imaginaríamos e implementar nosso próprio simulador entendendo seus defeitos e suas qualidades. Conseguimos aprimorar nossas habilidades como desenvolvedores, nossa programação em python e aprendemos a utilizar uma engine nova. Para o futuro, temos o desejo de continuar aprimorando nosso simulador, colocando mais funções, como colocar blocos, fluídos como água e lava, mais opções de geração de mundo, iluminação adequada e uma otimização melhor, utilizando os conhecimentos adquiridos com o Minecraft sobre chunks e face culling. Nosso simulador será de código aberto, permitindo que qualquer pessoa possa utilizá-lo e modificá-lo.

REFERÊNCIAS

BLATZ, Michael; KORN, Oliver. A Very Short History of Dynamic and Procedural Content Generation. 2017. Disponível em: https://www.researchgate.net/publication/315863952_A_Very_Short_History_of_Dynamic_and_Procedural_Content_Generation. Acesso em 14 de novembro de 2025.

CHECCHI, Tommaso. The Advanced Cave Culling Algorithm™, or, making Minecraft faster. 2014. Disponível em: https://tomcc.github.io/2014/08/31/visibility-1.html. Acesso em 11 de novembro de 2025.

CHECCHI, Tommaso. The Advanced Cave Culling Algorithm™ Part 2 - Traversing the graph. 2014. Disponível em: https://tomcc.github.io/2014/08/31/visibility-2.html. Acesso em 11 de novembro de 2025.

CRAFTWORKGAMES. Infiminer. Código fonte. 2009. Disponível em: https://github.com/craftworkgames/infiniminer. Acesso em 18 de outubro de 2025.

Documentário How Minecraft ACTUALLY Works. Vídeo (46:01). Publicado pelo canal Alan Zucconi. 2022. Disponível em: https://www.youtube.com/watch?v=YyVAaJqYAfE. Acesso em 10 de novembro de 2025.

Game Developer. A tecnologia por trás de Minecraft, 2011. Disponível em: https://gamedeveloper.com.br/a-tecnologia-por-tras-de-minecraft/. Acesso em 12 de novembro de 2025.

Kenny. Procedural Generation: An Overview. 2021. Disponível em: https://kentpawson123.medium.com/procedural-generation-an-overview-1b054a0f8d41. Acesso em 14 de novembro de 2025.

Lmas. Opensimplex. 2024. Disponível em: https://pypi.org/project/opensimplex/. Acesso em 12 de novembro de 2025.

MELNYCHUK, Vladyslav. Landscape generation using procedural generation techniques. 2020. Disponível em: https://scholar.google.com/scholar?hl=pt-BR&as_sdt=0%2C5&q=Landscape+generation+using+procedural+generation+techniques+vladyslav&btnG=. Acesso em 12 de novembro de 2025.

Minecraft Wiki. Chunk. 2025. Disponível em: https://minecraft.wiki/w/Chunk. Acesso em 12 de novembro de 2025.

Minecraft Wiki. Geração de mundo. 2025. Disponível em: https://pt.minecraft.wiki/w/Gera%C3%A7%C3%A3o_de_mundo. Acesso em 12 de novembro de 2025.

MobyGames. Infiniminer Windows Screenshots. 2009. Disponível em: https://www.mobygames.com/game/214999/infiniminer/screenshots/. Acesso em 18 de outubro de 2025.

Nathan Young. Apex Hosting. How to Show Chunk Borders in Minecraft. 2022. Disponível em: https://apexminecrafthosting.com/how-to-show-chunk-borders-in-minecraft. Acesso em 16 de novembro de 2025.

PokePetter. Documentação do Ursina. 2020. Disponível em: https://www.ursinaengine.org/documentation.html. Acesso em 18 de outubro de 2025.

ZUCCONI, Alan. The World Generation of Minecraft. Alanzucconi, 2022. Disponível em: https://www.alanzucconi.com/2022/06/05/minecraft-world-generation/. Acesso em 10 de novembro de 2025.